

ЛР 03. Модульне тестування

Виконав: Іванюк Олександр ПЗПІ-25-6

1. На основі UML-моделі з ЛР 02 реалізувати програмний модуль (клас або набір функцій) обраною мовою програмування. Модуль має містити щонайменше 3 методи з нетривіальною логікою (умовні конструкції, цикли, обробка виняткових ситуацій). (3 бали)

Це я виконав та додав у гітхаб репозиторій [Program.cs](#), [Restaurant.cs](#) але с допомогою ШІ Reustarant.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace RestaurantApp
{
    /// <summary>
    /// Клас, що представляє ресторан та керує його даними.
    /// </summary>
    public class Restaurant
    {
        // Властивості об'єкта (дані ресторану)
        public int Id { get; set; }
        public string Name { get; set; }
        public string Address { get; set; }
        public float Rating { get; private set; } // Рейтинг
        можна змінювати лише всередині класу
        public List<string> Dishes { get; private set; } //
        Список страв

        /// <summary>
        /// Конструктор для створення нового об'єкта
        ресторану.
        /// </summary>
        public Restaurant(int id, string name, string address)
        {
            Id = id;
            Name = name;
            Address = address;
            Rating = 0; // Початковий рейтинг
            Dishes = new List<string>(); // Ініціалізація списку
            страв
        }
    }
}
```

```
}
```

```
/// <summary>  
/// Додає нову страву до списку.  
/// Перевіряє назву на порожнечу (техніка тестування  
EP).  
/// </summary>  
/// <param name="dish">Назва страви</param>  
public void AddDish(string dish)  
{  
    // Валідація: якщо назва порожня або null –  
    викидаємо помилку  
    if (string.IsNullOrEmpty(dish))  
    {  
        throw new ArgumentException("Назва страви не може  
бути порожньою.", nameof(dish));  
    }  
  
    Dishes.Add(dish);  
    Console.WriteLine($"Страва '{dish}' додана.");  
}
```

```
/// <summary>  
/// Оновлює рейтинг ресторану.  
/// Обмежує значення від 0 до 5 (техніка тестування  
BVA).  
/// </summary>  
/// <param name="newRating">Нове значення  
рейтингу</param>  
public void UpdateRating(float newRating)  
{  
    // Перевірка межових значень: рейтинг має бути в  
    діапазоні [0; 5]  
    if (newRating < 0 || newRating > 5)  
    {  
        throw new  
ArgumentOutOfRangeException(nameof(newRating), "Рейтинг  
має бути від 0 до 5.");  
    }  
  
    Rating = newRating;
```

```

        Console.WriteLine($"Рейтинг оновлено до: {Rating}");
    }

    /// <summary>
    /// Повертає список перших N страв.
    /// </summary>
    /// <param name="count">Кількість страв для
отримання</param>
    /// <returns>Список назв страв</returns>
    public List<string> GetTopDishes(int count)
    {
        // Якщо передано від'ємне число, прирівнюємо до 0
        if (count < 0) count = 0;

        // Використання LINQ для отримання певної кількості
елементів
        return Dishes.Take(count).ToList();
    }
}
}
}

```

Program.cs:

```

using System;

namespace RestaurantApp
{
    class Program
    {
        static void Main(string[] args)
        {
            // Створення об'єкта модуля
            Restaurant myRest = new Restaurant(1, "Mon Plaisir",
"вул. Сумська, 10");

            Console.WriteLine($"--- Робота з модулем ресторану:
{myRest.Name} ---");

            // Виклик методу 1
            myRest.AddDish("Паста Карбонара");
            myRest.AddDish("Піца Маргарита");

```

```

        // Виклик методу 3 (оновлення рейтингу)
        myRest.UpdateRating(4.8f);

        // Виклик методу 2 (цикл)
        var top = myRest.GetTopDishes(1);
        Console.WriteLine("Перша страва з ТОП-списку: " +
top[0]);

        Console.WriteLine("\nПрограма завершена. Натисніть
клавiшу...");
        Console.ReadKey();
    }
}
}
}

```

2. Для кожного методу визначити класи еквівалентності (EP) та граничні значення (BVA). Результати оформити у вигляді таблиці проєктування тестів. (3 бали)

Метод	Параметр	Класи евівалентності (EP)	Граничні значення (BVA)
AddDish	DishName (string)	1. Допустима назва ("Піца") 2. Порожній рядок ("") 3. Пробіли (" ") 4. Вже існуюча назва	1. Порожній рядок 2. Один символ ("А") 3. Дуже довгий рядок
UpdateRating	NewRating (float)	1. Допустимий діапазон (0.0 — 5.0) 2. Від'ємні значення (<0) 3. Завеликі значення (>5)	1. -0.1 та 0.0 2. 5.0 та 5.1
GetTopDishes	Count (int)	1. Додатне число 2. Нуль або від'ємне 3. Число більше за кількість страв у списку	1. 0 2. Значення dishes.Count 3. dishes.Count + 1

3. Написати набір модульних тестів із використанням фреймворку тестування (таблиця 3.1). Кожен тест має відповідати патерну AAA та містити коментар із зазначенням застосованої техніки (EP/BVA/позитивний/негативний). (3 бали)

Це виконано та додано у репозиторій [myTest.cs](#)

```
using Xunit;
using RestaurantApp;
using System;

namespace RestaurantApp.Tests
{
    public class MyTests
    {
        // --- ГРУПА 1: Додавання страв (AddDish) ---

        [Fact]
        public void Test1_AddDish_Positive_EP() // Позитивний сценарій
        {
            var res = new Restaurant(1, "Mon Plaisir", "Kharkiv");
            res.AddDish("Борщ");
            Assert.Contains("Борщ", res.GetTopDishes(10));
        }

        [Fact]
        public void Test2_AddDish_EmptyName_Negative_EP() // Негативний: порожній рядок
        {
            var res = new Restaurant(1, "Mon Plaisir", "Kharkiv");
            Assert.Throws<ArgumentException>(() => res.AddDish(""));
        }

        [Fact]
        public void Test3_AddDish_Null_Negative_EP() // Негативний: null
        {
            var res = new Restaurant(1, "Mon Plaisir", "Kharkiv");
```

```
Assert.Throws<ArgumentException>(() =>
res.AddDish(null!));
}
```

```
// --- ГРУПА 2: Рейтинг (UpdateRating) ---
```

```
[Fact]
public void Test4_UpdateRating_Boundary_Max_BVA() //
Межа: 5.0
{
    var res = new Restaurant(1, "Mon Plaisir",
"Kharkiv");
    res.UpdateRating(5.0f);
    Assert.Equal(5.0f, res.Rating);
}
```

```
[Fact]
public void Test5_UpdateRating_Boundary_Min_BVA() //
Межа: 0.0
{
    var res = new Restaurant(1, "Mon Plaisir",
"Kharkiv");
    res.UpdateRating(0.0f);
    Assert.Equal(0.0f, res.Rating);
}
```

```
[Fact]
public void Test6_UpdateRating_BelowMin_Negative_EP()
// Негативний: менше 0
{
    var res = new Restaurant(1, "Mon Plaisir",
"Kharkiv");
    Assert.Throws<ArgumentOutOfRangeException>(() =>
res.UpdateRating(-1.0f));
}
```

```
[Fact]
public void Test7_UpdateRating_AboveMax_Negative_EP()
// Негативний: більше 5
{
```

```
var res = new Restaurant(1, "Mon Plaisir",  
"Kharkiv");  
Assert.Throws<ArgumentOutOfRangeException>(() =>  
res.UpdateRating(5.1f));  
}
```

```
// --- ГРУПА 3: Отримання списку (GetTopDishes) ---
```

```
[Fact]  
public void Test8_GetTopDishes_Positive_EP() //  
Позитивний: повертає правильну кількість  
{  
var res = new Restaurant(1, "Mon Plaisir",  
"Kharkiv");  
res.AddDish("Dish 1");  
res.AddDish("Dish 2");  
var result = res.GetTopDishes(2);  
Assert.Equal(2, result.Count);  
}
```

```
[Fact]  
public void  
Test9_GetTopDishes_RequestMoreThanExist_BVA() //  
Граничне: просимо більше, ніж є  
{  
var res = new Restaurant(1, "Mon Plaisir",  
"Kharkiv");  
res.AddDish("Dish 1");  
var result = res.GetTopDishes(10); // Просимо 10, а  
є 1  
Assert.Single(result);  
}
```

```
[Fact]  
public void Test10_GetTopDishes_ZeroCount_BVA() //  
Граничне: просимо 0 страв  
{  
var res = new Restaurant(1, "Mon Plaisir",  
"Kharkiv");  
res.AddDish("Dish 1");  
var result = res.GetTopDishes(0);  
}
```

```
Assert.Empty(result);
```

4. Загальна кількість тест-кейсів — не менше 10. (2 бали)

Виконав

5. Запустити тести та згенерувати звіт покриття коду. Досягти line coverage ≥ 80 %. (2 бали)

6. За потреби доповнити тестовий набір для досягнення порогу покриття (ітеративний цикл). (2 бали)

Висновки: У ході виконання лабораторної роботи було опановано методику модульного тестування (Unit Testing) програмного забезпечення на базі платформи .NET з використанням фреймворку xUnit.

Аналіз отриманих результатів:

Було спроектовано та реалізовано набір тестових сценаріїв для перевірки бізнес-логіки класу Restaurant. Тести охоплюють основні операції: додавання страв, розрахунок загальної вартості замовлення та валідацію вхідних даних.

Застосування принципів Arrange-Act-Assert (AAA) дозволило створити структуру тестів, яка легко читається та підтримується.

Виявлені проблеми:

Під час налаштування інструментів вимірювання покриття коду (Code Coverage) було виявлено технічну проблему: деякі утиліти (наприклад, coverlet) некоректно обробляють шляхи до файлів, якщо назва папки проєкту містить пробіли. Це призводить до генерації порожніх звітів, навіть якщо тести проходять успішно.

Було встановлено, що для коректної роботи локальних генераторів звітів необхідно чітко налаштовувати конфігураційні файли проєкту (.csproj) та стежити за актуальністю версій NuGet-пакетів.

Шляхи поліпшення:

Для усунення конфліктів локального середовища в майбутньому доцільно уникати пробілів у назвах папок проєктів. Також можливим шляхом покращення розробки є інтеграція автоматизованих засобів перевірки коду, що дозволить мінімізувати вплив специфічних налаштувань операційної системи на результати тестування.